Structural Modeling in Verilog HDL

Digital Design and RTL Fundamentals

October 12, 2026

Contents

1 Introduction **3**

2 Basic Structure of a Verilog Module **3**
2.1 General Syntax . . . 3

3 Gate-Level Structural Modeling **4**
3.1 Syntax . . . 4
3.2 Example: AND Gate . . . 4
3.3 Truth Table . . . 4

4 Examples of Built-In Gate Primitives **5**
4.1 OR Gate . . . 5
4.2 NOT Gate . . . 5
4.3 NAND Gate . . . 5
4.4 NOR Gate . . . 5
4.5 XOR Gate . . . 6
4.6 XNOR Gate . . . 6

5 Multiple Gates in a Structural Design **6**
5.1 Example: AND-OR Circuit . . . 6

6 Structural Modeling of a Half Adder **7**
6.1 Verilog Code . . . 7
6.2 Truth Table . . . 7

7 Structural Modeling of a Full Adder **8**
7.1 Verilog Code . . . 8

8 Module Instantiation **9**
8.1 Positional Port Connection . . . 9
8.2 Named Port Connection . . . 9

9 Structural Modeling of a Half Adder Using Module Instantiation **10**
9.1 XOR Module . . . 10
9.2 AND Module . . . 10
9.3 Top-Level Half Adder . . . 10

10 Structural Modeling of a 2-to-1 Multiplexer **11**
10.1 Verilog Code 11
10.2 Operation 11

11 Structural Modeling of a Decoder **11**
11.1 Verilog Code 12
11.2 Truth Table 12

12 Structural Modeling of a 4-Bit Ripple Carry Adder **12**
12.1 Verilog Code 12

13 Testbench for Structural Modeling **14**
13.1 Example: Half Adder Testbench 14

14 Structural Modeling vs. Dataflow Modeling **14**

15 Structural Modeling vs. Behavioral Modeling **15**

16 Advantages of Structural Modeling **15**

17 Limitations of Structural Modeling **15**

18 Common Errors and Best Practices **15**

19 Structural Modeling Summary **16**

20 Summary **16**

1 Introduction

Structural modeling is a design modeling style in Verilog HDL in which a digital circuit is described by connecting smaller hardware components together.

These components may be built-in logic gates, user-defined modules, or other supported hardware primitives.

Structural modeling describes the structure and interconnections of a circuit rather than describing its behavior through procedural statements.

It is commonly used for gate-level modeling, hierarchical design, and connecting modules to construct larger digital systems.

The main features of structural modeling are:

- Instantiation of gates and modules.
- Explicit connections between components.
- Use of wires to carry signals.
- Hierarchical construction of circuits.
- Reuse of smaller modules in larger designs.

2 Basic Structure of a Verilog Module

A structural Verilog design typically contains a module declaration, input and output ports, internal nets, and component instantiations.

2.1 General Syntax

```
module module_name (
    input a,
    input b,
    output y
);

    wire internal_signal;

    // Component instantiations go here

endmodule
```

The module defines the circuit interface. Internal wires connect the outputs of some components to the inputs of other components.

A structural module can contain several instantiated components connected through nets.

3 Gate-Level Structural Modeling

Verilog provides built-in gate primitives that can be instantiated directly.

Common gate primitives include:

- `and` – AND gate
- `or` – OR gate
- `not` – NOT gate
- `nand` – NAND gate
- `nor` – NOR gate
- `xor` – XOR gate
- `xnor` – XNOR gate
- `buf` – Buffer

3.1 Syntax

```
and instance_name (output_signal, input1, input2);
```

For gate primitives, the output terminal is listed first, followed by the input terminals.

3.2 Example: AND Gate

```
module and_gate (
    input a,
    input b,
    output y
);

    and g1 (y, a, b);

endmodule
```

The instance `g1` represents an AND gate. The output `y` is 1 only when both inputs are 1.

3.3 Truth Table

a	b	y
0	0	0
0	1	0
1	0	0
1	1	1

4 Examples of Built-In Gate Primitives

4.1 OR Gate

```
module or_gate (
    input a,
    input b,
    output y
);
    or g1 (y, a, b);
endmodule
```

The OR gate output is 1 when at least one input is 1.

4.2 NOT Gate

```
module not_gate (
    input a,
    output y
);
    not g1 (y, a);
endmodule
```

The NOT gate produces the complement of its input.

4.3 NAND Gate

```
module nand_gate (
    input a,
    input b,
    output y
);
    nand g1 (y, a, b);
endmodule
```

The NAND gate is the complement of the AND gate.

4.4 NOR Gate

```
module nor_gate (
    input a,
    input b,
    output y
);
    nor g1 (y, a, b);
endmodule
```

The NOR gate is the complement of the OR gate.

4.5 XOR Gate

```
module xor_gate (
    input a,
    input b,
    output y
);
    xor g1 (y, a, b);
endmodule
```

The XOR gate output is 1 when the inputs differ.

4.6 XNOR Gate

```
module xnor_gate (
    input a,
    input b,
    output y
);
    xnor g1 (y, a, b);
endmodule
```

The XNOR gate output is 1 when the inputs are equal.

5 Multiple Gates in a Structural Design

A circuit can be created by connecting the outputs of multiple gates to other gates.

5.1 Example: AND-OR Circuit

Consider the Boolean function:

$$Y = (A \wedge B) \vee C$$

The circuit requires one AND gate and one OR gate.

```
module and_or_circuit (
    input A,
    input B,
    input C,
    output Y
);

    wire w1;

    and g1 (w1, A, B);
    or  g2 (Y, w1, C);

endmodule
```

Explanation:

- The AND gate generates `w1 = A & B`.
- The OR gate combines `w1` and `C`.
- The final output is `Y = (A & B) | C`.

The internal wire `w1` connects the output of the first gate to the second gate.

6 Structural Modeling of a Half Adder

A half adder adds two single-bit binary inputs and generates a sum and a carry output.

The Boolean equations are:

$$S = A \oplus B$$

$$C = A \wedge B$$

where:

- A and B are input bits.
- S is the sum output.
- C is the carry output.

6.1 Verilog Code

```
module half_adder (
    input A,
    input B,
    output S,
    output C
);

    xor g1 (S, A, B);
    and g2 (C, A, B);

endmodule
```

6.2 Truth Table

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

7 Structural Modeling of a Full Adder

A full adder adds three single-bit inputs: A, B, and carry-in C_{in}.

It produces a sum S and carry-out C_{out}.

The Boolean equations are:

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$

7.1 Verilog Code

```
module full_adder (
    input A,
    input B,
    input Cin,
    output S,
    output Cout
);

    wire w1, w2, w3;

    xor g1 (w1, A, B);
    xor g2 (S, w1, Cin);

    and g3 (w2, A, B);
    and g4 (w3, w1, Cin);

    or  g5 (Cout, w2, w3);

endmodule
```

The carry equation used in this implementation is equivalent to:

$$C_{out} = AB + (A \oplus B)C_{in}$$

The two AND gates generate carry terms, and the OR gate combines them.

8 Module Instantiation

Structural modeling also allows one module to instantiate another module.

This is called module instantiation and is the foundation of hierarchical design.

8.1 Positional Port Connection

In positional connection, ports are connected according to their order in the module declaration.

```
module inverter (
    input a,
    output y
);

    not g1 (y, a);

endmodule

module top_module (
    input x,
    output z
);

    inverter u1 (x, z);

endmodule
```

In this example, the connections follow the port order declared in the `inverter` module.

For clarity, named port connections are often preferred in larger designs.

8.2 Named Port Connection

```
module top_module (
    input x,
    output z
);

    inverter u1 (
        .a(x),
        .y(z)
    );

endmodule
```

Named connections explicitly associate each instance port with a signal in the parent module.

They reduce the risk of accidentally connecting signals in the wrong order.

9 Structural Modeling of a Half Adder Using Module Instantiation

A half adder can be constructed by instantiating separate XOR and AND modules.

9.1 XOR Module

```
module xor_module (
    input a,
    input b,
    output y
);

    xor g1 (y, a, b);

endmodule
```

9.2 AND Module

```
module and_module (
    input a,
    input b,
    output y
);

    and g1 (y, a, b);

endmodule
```

9.3 Top-Level Half Adder

```
module half_adder_top (
    input A,
    input B,
    output S,
    output C
);
    xor_module u1 (
        .a(A),
        .b(B),
        .y(S)
    );

    and_module u2 (
        .a(A),
        .b(B),
        .y(C)
    );
endmodule
```

This design uses two smaller modules to construct a larger circuit.

The modules can be reused in other designs without rewriting their internal implementations.

10 Structural Modeling of a 2-to-1 Multiplexer

A 2-to-1 multiplexer selects one of two inputs based on a select signal.

The Boolean equation is:

$$Y = \overline{S}A + SB$$

where S is the select input.

10.1 Verilog Code

```
module mux2to1 (
    input A,
    input B,
    input S,
    output Y
);

    wire nS, w1, w2;

    not g1 (nS, S);
    and g2 (w1, A, nS);
    and g3 (w2, B, S);
    or  g4 (Y, w1, w2);

endmodule
```

10.2 Operation

S	Output Y
0	A
1	B

When $S = 0$, the output follows A. When $S = 1$, the output follows B.

11 Structural Modeling of a Decoder

A 2-to-4 decoder converts a two-bit input into one of four active-high outputs.

Each output corresponds to one input combination.

11.1 Verilog Code

```
module decoder2to4 (
    input A,
    input B,
    output [3:0] Y
);

    wire nA, nB;

    not g1 (nA, A);
    not g2 (nB, B);

    and g3 (Y[0], nA, nB);
    and g4 (Y[1], nA, B);
    and g5 (Y[2], A, nB);
    and g6 (Y[3], A, B);

endmodule
```

11.2 Truth Table

A	B	Y[3]	Y[2]	Y[1]	Y[0]	Selected output
0	0	0	0	0	1	Y[0]
0	1	0	0	1	0	Y[1]
1	0	0	1	0	0	Y[2]
1	1	1	0	0	0	Y[3]

Only one output is active for each valid binary input combination.

12 Structural Modeling of a 4-Bit Ripple Carry Adder

A ripple carry adder can be constructed by connecting four full adders in sequence.

The carry output of each stage becomes the carry input of the next stage.

12.1 Verilog Code

```
module ripple_carry_adder (
    input [3:0] A,
    input [3:0] B,
    input Cin,
    output [3:0] Sum,
    output Cout
);

    wire c1, c2, c3;

    full_adder fa0 (
        .A(A[0]),
        .B(B[0]),
```

```
        .Cin(Cin),
        .S(Sum[0]),
        .Cout(c1)
    );

    full_adder fa1 (
        .A(A[1]),
        .B(B[1]),
        .Cin(c1),
        .S(Sum[1]),
        .Cout(c2)
    );

    full_adder fa2 (
        .A(A[2]),
        .B(B[2]),
        .Cin(c2),
        .S(Sum[2]),
        .Cout(c3)
    );

    full_adder fa3 (
        .A(A[3]),
        .B(B[3]),
        .Cin(c3),
        .S(Sum[3]),
        .Cout(Cout)
    );

endmodule
```

The module requires the `full_adder` definition from the earlier section to be compiled as part of the same design.

The carry propagates from the least significant bit to the most significant bit.

13 Testbench for Structural Modeling

A testbench applies input combinations and observes the outputs of a design.

It is generally written using procedural constructs, even when the design under test uses structural modeling.

13.1 Example: Half Adder Testbench

```
module half_adder_tb;

    reg A, B;
    wire S, C;

    half_adder dut (
        .A(A),
        .B(B),
        .S(S),
        .C(C)
    );

    initial begin
        $monitor("Time=%0t A=%b B=%b S=%b C=%b",
                 $time, A, B, S, C);

        A = 0; B = 0;
        #10 A = 0; B = 1;
        #10 A = 1; B = 0;
        #10 A = 1; B = 1;
        #10 $finish;
    end

endmodule
```

The testbench applies all four possible input combinations and displays the sum and carry outputs.

The `half_adder` module must be compiled along with the testbench.

14 Structural Modeling vs. Dataflow Modeling

Feature	Structural Modeling	Dataflow Modeling
Description	Components and their connections.	Boolean expressions using continuous assignments.
Main constructs	Gate and module instantiations.	`assign` statements.
Circuit structure	Explicitly represented.	Inferred from expressions.
Example	`and g1(y,a,b);`	`assign y = a & b;`
Typical use	Gate-level and hierarchical designs.	Boolean logic and combinational equations.

15 Structural Modeling vs. Behavioral Modeling

Feature	**Structural Modeling**	**Behavioral Modeling**
Description	Circuit components and interconnections.	Operations and procedural behavior.
Main constructs	Gate and module instances.	`always` and `initial` blocks.
Abstraction	Usually lower-level.	Often higher-level.
Example	Connecting gates to form an adder.	Using an expression or procedural logic to describe an adder.
Typical use	Gate-level implementation and hierarchy.	RTL algorithms and control logic.

16 Advantages of Structural Modeling

- Makes component connections explicit.
- Supports hierarchical design.
- Allows modules to be reused.
- Helps describe gate-level circuits.
- Can represent a circuit using interconnected logic primitives.
- Is useful for understanding the organization of digital hardware.

17 Limitations of Structural Modeling

- Large circuits may require many instances and internal wires.
- Source code can become lengthy.
- Manual wiring can introduce connection errors.
- Changes to a large circuit may require many structural edits.
- Complex control algorithms are often easier to express behaviorally.
- Gate-level descriptions do not automatically guarantee a particular physical implementation after synthesis.

18 Common Errors and Best Practices

Common errors include:

- Connecting module ports in the wrong positional order.
- Using a signal name that has not been declared.
- Forgetting to declare internal wires.
- Connecting a gate output to an inappropriate net or port.
- Forgetting to compile a module definition required by another module.

- Creating unintended multiple drivers.

Recommended practices include:

- Use named port connections for readability.
- Give instances meaningful names.
- Declare internal nets explicitly.
- Divide complex circuits into smaller reusable modules.
- Verify gate-level behavior using a testbench.
- Check port widths and signal directions.
- Keep module interfaces consistent.

19 Structural Modeling Summary

Construct	**Purpose**
`and`	Instantiates an AND gate primitive.
`or`	Instantiates an OR gate primitive.
`not`	Instantiates a NOT gate primitive.
`nand`	Instantiates a NAND gate primitive.
`nor`	Instantiates a NOR gate primitive.
`xor`	Instantiates an XOR gate primitive.
`xnor`	Instantiates an XNOR gate primitive.
`wire`	Declares a net used to connect components.
Module instantiation	Connects an instance of one module to another.
Named port connection	Connects ports using explicit port names.
Positional port connection	Connects ports according to their declared order.

20 Summary

- Structural modeling describes a circuit using interconnected components.
- Built-in gate primitives represent basic logic gates.
- Internal wires connect the outputs and inputs of components.
- Module instantiation enables hierarchical and reusable design.
- Half adders, full adders, multiplexers, decoders, and ripple carry adders can be described structurally.
- Testbenches are used to verify the behavior of structural designs.
- Structural modeling is useful for gate-level design and understanding circuit interconnections.

Key takeaway: Structural modeling describes how a digital circuit is built by connecting gates and modules. It is especially useful for hierarchical design, gate-level descriptions, and reusable hardware components.